BTS CIEL · PROGRAMMATION C++ · COURS

Chapitre 4 — Boucles, tableaux et conteneurs

Prérequis : chapitres 1 à 3 · **Exercices :** `E04_Boucles_tableaux_conteneurs`

Objectifs

- répéter un traitement avec `while`, `for` et `do … while`, et contrôler une boucle avec `break` et `continue` ;
- stocker des valeurs dans un tableau, un `std::array` ou un `std::vector`, et les parcourir ;
- effectuer des calculs sur une collection (somme, moyenne, maximum, recherche) ;
- lire un tableau à deux dimensions ;
- redemander une saisie tant qu'elle n'est pas valide.

Sommaire

- Chapitre 4 — Boucles, tableaux et conteneurs
  - Objectifs
  - Sommaire
  - 1. Les boucles
    - 1.1 `break` et `continue`
  - 2. Les tableaux classiques
  - 3. Effectuer des calculs sur un tableau
    - 3.1 Calculer une moyenne
    - 3.2 Rechercher une valeur maximale
  - 4. Les conteneurs de la bibliothèque standard
    - 4.1 `std::array`
    - 4.2 `std::vector`
    - 4.3 Indices et type `std::size_t`
    - 4.4 La boucle `for` sur un intervalle
  - 5. Les tableaux à deux dimensions
  - 6. Saisie contrôlée en boucle
  - Erreurs fréquentes
  - Synthèse

1. Les boucles

```cpp
int compteur{1};

// Boucle while
while (compteur <= 5) {          // condition testee AVANT chaque passage
    std::cout << compteur << '\n';
    compteur++;                  // sans mise a jour : boucle infinie
}

// Boucle for
for (int i = 1; i <= 5; i++) {   // initialisation ; condition ; mise a jour
    std::cout << i << '\n';
}

// Boucle do ... while
int choix{};
do {                             // execute AU MOINS une fois
    std::cout << "Tapez 0 pour quitter : ";
    std::cin >> choix;
} while (choix != 0);            // point-virgule obligatoire
```

Situation	Boucle conseillée
Nombre de répétitions connu, parcours d'une collection	`for`
Répéter tant qu'une condition est vraie	`while`
Exécuter au moins une fois (menu, saisie)	`do … while`

Dans un `for`, la variable `i` n'existe qu'à l'intérieur de la boucle.

1.1 `break` et `continue`

- `break` **quitte** immédiatement la boucle ;
- `continue` **passe** directement à l'itération suivante, sans exécuter la fin du bloc.

```cpp
#include <iostream>

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;            // nombre pair : on passe au suivant
        }
        if (i > 7) {
            break;               // au-dela de 7 : on quitte la boucle
        }
        std::cout << i << ' ';
    }
    std::cout << '\n';

    return 0;
}
```

Résultat :

```
1 3 5 7
```

2. Les tableaux classiques

Un **tableau** stocke un nombre **fixe** de valeurs de même type, accessibles par un **indice** de `0` à `taille - 1`. On nomme la taille par une constante plutôt que de répéter le nombre.

```cpp
#include <iostream>

int main() {
    constexpr int NB_MESURES{5};
    int mesures[NB_MESURES]{12, 15, 18, 20, 22};   // aussi : int t[10]{}; (tout a 0)

    mesures[1] = 25;

    for (int i = 0; i < NB_MESURES; i++) {
        std::cout << mesures[i] << ' ';
    }
    std::cout << '\n';

    return 0;
}
```

Résultat :

```
12 25 18 20 22
```

> **Attention :** `mesures[5]` est hors limites : c'est un **comportement indéfini**, et le compilateur ne vérifie pas l'indice.

3. Effectuer des calculs sur un tableau

3.1 Calculer une moyenne

```cpp
#include <iostream>

int main() {
    constexpr int NB_VALEURS{4};
    int valeurs[NB_VALEURS]{10, 20, 30, 40};
    int somme{};

    for (int i = 0; i < NB_VALEURS; i++) {
        somme += valeurs[i];
    }

    double moyenne{static_cast<double>(somme) / NB_VALEURS};

    std::cout << "Moyenne : " << moyenne << '\n';

    return 0;
}
```

Résultat :

```
Moyenne : 25
```

La conversion en `double` permet d'obtenir une division réelle.

3.2 Rechercher une valeur maximale

```cpp
#include <iostream>

int main() {
    constexpr int NB_VALEURS{5};
    int valeurs[NB_VALEURS]{12, 8, 25, 17, 10};

    int maximum{valeurs[0]};      // on part du premier element

    for (int i = 1; i < NB_VALEURS; i++) {
        if (valeurs[i] > maximum) {
            maximum = valeurs[i];
        }
    }

    std::cout << "Maximum : " << maximum << '\n';

    return 0;
}
```

Résultat :

```
Maximum : 25
```

> **Attention :** si `maximum` était initialisé à `0` et que toutes les valeurs étaient négatives, le résultat serait faux. On initialise avec le **premier élément**.

4. Les conteneurs de la bibliothèque standard

Conteneur	En-tête	Taille	Usage typique
`std::array<T, N>`	`<array>`	Fixe	Nombre de valeurs connu (4 octets d'une adresse IPv4)
`std::vector<T>`	`<vector>`	Variable	Nombre de valeurs inconnu (mesures saisies, capteurs détectés)

Contrairement au tableau classique, ils connaissent leur taille (`size()`) et proposent `at(i)` , qui vérifie l'indice.

4.1 `std::array`

```cpp
#include <array>
#include <iostream>

int main() {
    std::array<int, 4> octets{192, 168, 1, 10};

    std::cout << "Nombre d'octets : " << octets.size() << '\n';
    std::cout << "Premier octet : " << octets[0] << '\n';

    return 0;
}
```

Résultat :

```
Nombre d'octets : 4
Premier octet : 192
```

4.2 `std::vector`

Un `std::vector` peut grandir : `push_back()` ajoute un élément à la fin.

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<double> mesures{};

    mesures.push_back(21.5);
    mesures.push_back(22.0);
    mesures.push_back(23.5);

    std::cout << "Nombre de mesures : " << mesures.size() << '\n';

    if (!mesures.empty()) {
        std::cout << "Derniere mesure : " << mesures.back() << '\n';
    }

    return 0;
}
```

Résultat :

```
Nombre de mesures : 3
Derniere mesure : 23.5
```

Méthode	Rôle
`push_back(v)`	Ajoute `v` à la fin
`size()`	Nombre d'éléments
`empty()`	`true` si le vecteur est vide
`v[i]`	Élément d'indice `i`, sans vérification
`at(i)`	Élément d'indice `i`, avec vérification
`back()`	Dernier élément (le vecteur ne doit pas être vide)
`clear()`	Supprime tous les éléments

Remarque : si l'indice est invalide, `at(i)` **lève une exception** (`std::out_of_range`) : sans traitement particulier, le programme s'arrête avec un message d'erreur, au lieu de lire une valeur au hasard. Le traitement des exceptions est présenté au chapitre 8 (§6).

4.3 Indices et type `std::size_t`

`size()` renvoie une valeur de type `std::size_t` : un entier **non signé**, prévu pour les tailles et les indices. Pour parcourir un conteneur avec un indice, on utilise ce type.

```cpp
#include <cstddef>
#include <iostream>
#include <vector>

int main() {
    std::vector<int> valeurs{10, 20, 30};

    for (std::size_t i = 0; i < valeurs.size(); i++) {
        std::cout << "Indice " << i << " : " << valeurs[i] << '\n';
    }

    return 0;
}
```

Résultat :

```
Indice 0 : 10
Indice 1 : 20
Indice 2 : 30
```

Avec `int i`, la comparaison `i < valeurs.size()` mélange un entier signé et un entier non signé : g++ (`-Wall`) affiche l'avertissement `comparison of integer expressions of different signedness`.

> **Piège :** un `std::size_t` ne peut pas être négatif. `0 - 1` y donne une **très grande** valeur, pas `-1`. On évite donc de décrémenter un indice non signé jusqu'en dessous de zéro.

4.4 La boucle `for` sur un intervalle

Pour parcourir **tous** les éléments d'un tableau, d'un `std::array` ou d'un `std::vector`, sans gérer d'indice :

```cpp
#include <iostream>
#include <vector>

int main() {
    std::vector<int> valeurs{10, 20, 30};
    int somme{0};

    for (int valeur : valeurs) {        // valeur prend chaque element a tour de role
        somme += valeur;
    }

    std::cout << "Somme : " << somme << '\n';

    return 0;
}
```

Résultat :

```
Somme : 60
```

Pour **modifier** les éléments, on utilise une référence (chapitre 5, §5) : `for (int& valeur : valeurs) { valeur *= 2; }`.

5. Les tableaux à deux dimensions

Un tableau 2D représente une **grille** : image, matrice de LED, plan de places. Le premier indice désigne la **ligne**, le second la **colonne**.

```cpp
#include <iostream>

int main() {
    constexpr int NB_LIGNES{5};
    constexpr int NB_COLONNES{5};

    int pixels[NB_LIGNES][NB_COLONNES]{     // 1 = allume, 0 = eteint
        {1, 1, 1, 1, 1},
        {0, 0, 1, 0, 0},
        {0, 0, 1, 0, 0},
        {0, 0, 1, 0, 0},
        {0, 0, 1, 0, 0}
    };

    for (int ligne = 0; ligne < NB_LIGNES; ligne++) {
        for (int colonne = 0; colonne < NB_COLONNES; colonne++) {
            std::cout << (pixels[ligne][colonne] == 1 ? '#' : '.');
        }
        std::cout << '\n';
    }

    return 0;
}
```

Résultat :

```
#####
..#..
..#..
..#..
..#..
```

L'opérateur **ternaire** `condition ? a : b` vaut `a` si la condition est vraie, `b` sinon.

6. Saisie contrôlée en boucle

Au chapitre 3 (§4), une saisie invalide arrêtait le programme. Pour **redemander** la valeur, il faut savoir remettre `std::cin` en état de lire après un échec :

Instruction	Rôle
`std::cin.clear()`	Efface l'état d'échec : le flux peut lire à nouveau
`std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n')`	Jette le reste de la ligne mal saisie, jusqu'au retour à la ligne inclus (`#include <limits>`)
`std::cin.eof()`	Vrai si l'entrée est fermée (Ctrl+D sous Linux, Ctrl+Z puis Entrée sous Windows) : plus rien ne pourra être lu

```cpp
#include <iostream>
#include <limits>

int main() {
    constexpr int TEMPERATURE_MIN{-40};
    constexpr int TEMPERATURE_MAX{85};

    int temperature{};
    bool saisieValide{false};

    while (!saisieValide) {
        std::cout << "Temperature (" << TEMPERATURE_MIN << " a "
                  << TEMPERATURE_MAX << ") : ";

        if (std::cin >> temperature) {                        // lecture reussie
            if (temperature >= TEMPERATURE_MIN && temperature <= TEMPERATURE_MAX) {
                saisieValide = true;
            } else {
                std::cout << "Valeur hors limites.\n";
            }
        } else if (std::cin.eof()) {                          // entree fermee
            std::cerr << "Erreur : fin de la saisie.\n";
            return 1;
        } else {                                              // saisie non numerique
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
            std::cout << "Saisie non numerique.\n";
        }
    }

    std::cout << "Mesure acceptee : " << temperature << '\n';

    return 0;
}
```

Exemple d'exécution (les saisies suivent les deux-points) :

```
Temperature (-40 a 85) : abc
Saisie non numerique.
Temperature (-40 a 85) : 120
Valeur hors limites.
Temperature (-40 a 85) : 25
Mesure acceptee : 25
```

Attention : l'ordre compte. Sans `clear()`, `ignore()` ne fait rien car le flux est encore en échec ; sans `ignore()`, la boucle relit indéfiniment le même `abc`. Et sans le test `eof()`, une entrée fermée provoquerait une boucle infinie.

Erreurs fréquentes

Erreur	Effet	Correction
Variable de boucle jamais mise à jour	Boucle infinie	Vérifier la mise à jour dans chaque `while`
`for (...);` ou `while (...);`	Le `;` forme une boucle vide	Pas de `;` après la parenthèse
`i <= taille` dans un parcours	Lecture hors limites (comportement indéfini)	`i < taille`
Maximum initialisé à `0`	Faux si toutes les valeurs sont négatives	Initialiser avec le premier élément
`int i` comparé à `v.size()`	Avertissement `different signedness`	`std::size_t i` ou boucle sur un intervalle
`back()` sur un vecteur vide	Comportement indéfini	Tester `empty()` avant
Saisie ratée non réparée	Boucle infinie ou lectures suivantes ignorées	`clear()` puis `ignore(…)`
`;` oublié après `while (…)` d'un `do … while`	Erreur de compilation	`} while (condition);`

Synthèse

- `for` pour un nombre connu de répétitions, `while` pour une condition, `do … while` pour au moins une exécution ; `break` quitte la boucle, `continue` passe au tour suivant.
- Indices de 0 à taille − 1 ; hors limites : comportement indéfini. On nomme la taille par une constante.
- `std::array` : taille fixe ; `std::vector` : taille variable (`push_back` , `size` , `empty` , `back`) ; `at(i)` vérifie l'indice.
- Les indices et tailles sont de type `std::size_t` (non signé) ; `for (T x : collection)` parcourt tout sans indice.
- Tableau 2D : `t[ligne][colonne]` , deux boucles imbriquées.
- Saisie contrôlée : tester la lecture ; en cas d'échec, `clear()` puis `ignore(…)` ; tester `eof()` pour éviter une boucle infinie.